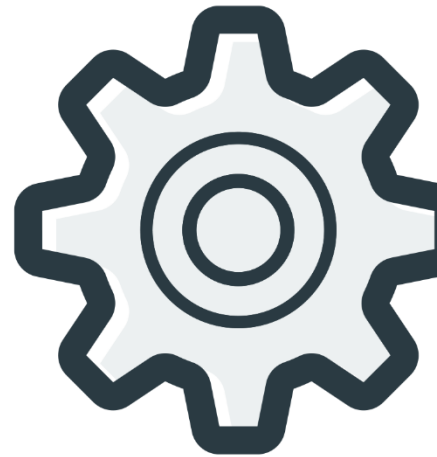
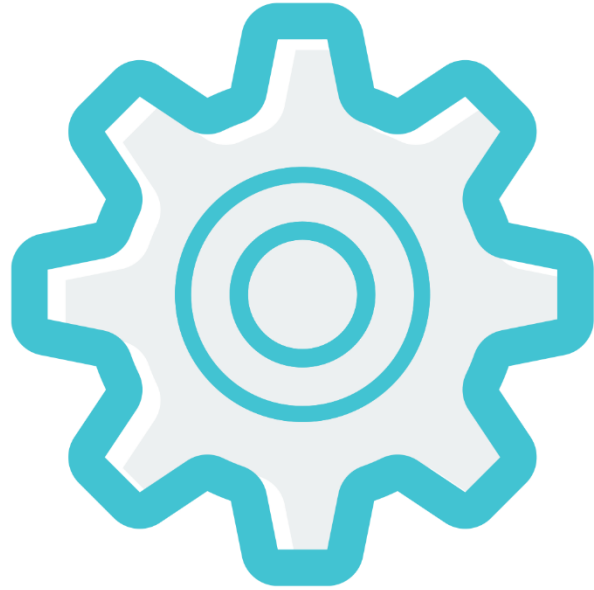


JUNIOR ROBOTICS

MANUAL

Path Finding Robot



AEROBOTICS GLOBAL

www.aeroboticsglobal.com

info@aeroboticsglobal.com

1800 237 627

CONTENTS

Important Information	2
Workshop 1	3-6
Workshop 2	7
Workshop 3	8-11
Workshop 4	12-14
Workshop 5 and 6	15-16
Workshop 7 and 8	17-19

WARNING: The kit contents are not suitable for children under 3 years. Choking hazard – small parts may be swallowed or inhaled.

Store the kit materials and assembled models out of reach of small children.



HANDLING AND PRECAUTIONS

General:

- Assembly and operation of this kit should be done only under the supervision of an adult
- All parts of this kit should be stored and operated away from water and fire sources
- All parts of this kit should never be put into the mouth
- Throwing or dropping the parts may cause damage to the parts

Building/Operating:

- All the tools for assembling must be used with extreme precaution
- The parts should not be forcibly bent, attached or removed
- The parts should not be touched when in motion
- The power supply, motors, sensors and other components should be checked before operating
- The power supply should be turned off when connecting the Arduino to the computer for programming

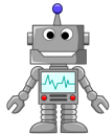


Batteries:

- The batteries should not be inserted in the battery holder before mounting and connecting the battery holder onto the model
- The batteries should be inserted correctly with the (+) and (-) in the correct direction
- Always use the provided batteries or batteries of correct size and rating (3.7V)
- New and used batteries should not be mixed for operation
- Always close the battery holder with the lid after removal or insertion of batteries

Workshop 1

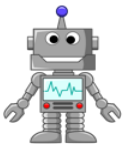
- Adding Liquid Crystal Display



Hi Andy, welcome to next part of the course which is to make a path finding robot. First, we start with adding an LCD display



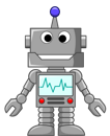
That is Exciting, what is an LCD display?



Liquid Crystal Display also known as LCD where you can display letters and number on the screen. LCD is extremely useful when you need to troubleshoot your robot. We are using a 16 x 2 Display, which means it have 16 columns and 2 rows where we can set the cursor at particular position, where we wanted to display the text or numbers



Okay, what is the first step?



First step is to assemble the display!

Step 1

Place two golden standoffs in the holes near the batter holder using the screws from the bottom of the robot frame

Refer to figures 1.02 and 1.03



Figure 1.01

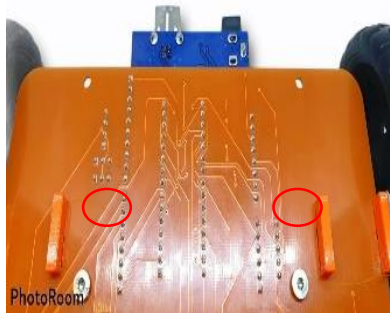


Figure 1.02



Figure 1.03

Follow these images to assemble the LCD display



Figure 1.04



Figure 1.05

Step 2

Connections:

Connect the LCD to the LCD pins highlighted in the board, make sure pins are connected correctly

- i) GND: ground or --
- ii) VCC: +
- iii) SDA - SDA
- iv) SCL - SCL

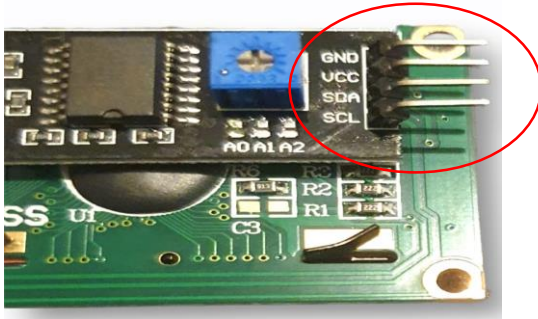


Figure 1.06

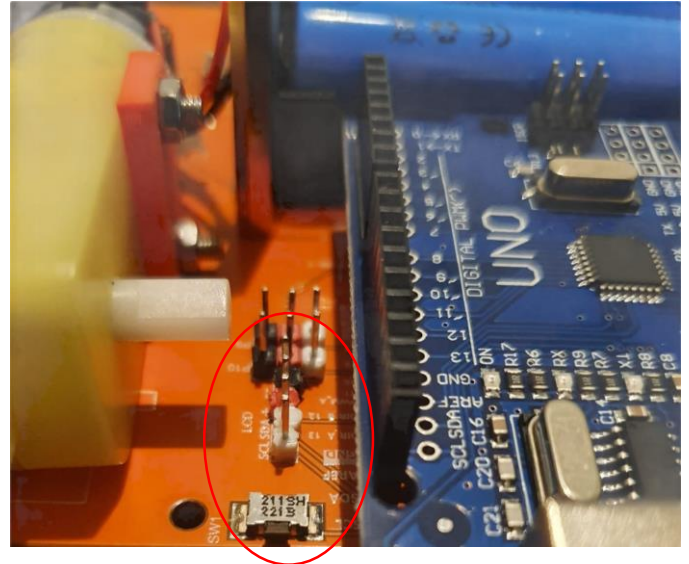


Figure 1.09

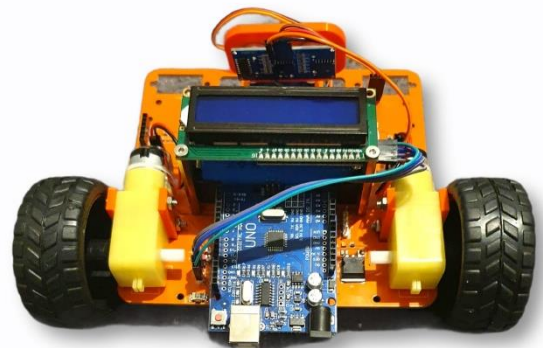


Figure 1.08

Step 3

After installation, we are ready to program the LCD

Before programming we need to add the Liquid Crystal Display zip file to our library, go to sketch and click on Include library and select add zip library and select our LCD file as shown in the below figure

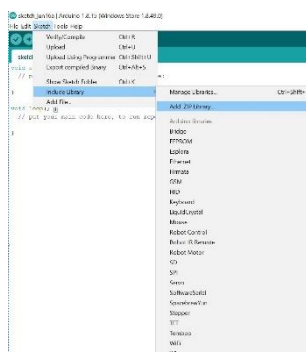


Figure 1.10

Address – The address represents the address of the LCD, we need this as we are using I2C protocol. There are two options available, we will use hit and try method, try both the address and one of them will work.

Let's make a program for our LCD to print "Hello World"

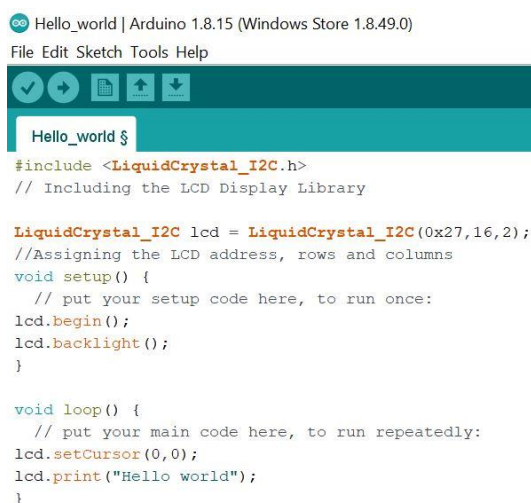


Figure 1.11

Challenge 1

Display your first name in the first row and last name in the second row

Workshop 2

Let us make a counter to count from 1 to 100 on LCD and display the same. We need to create a variable for this. Variable is like an empty container in which we can store different letters or numbers.



```

Hello_world | Arduino 1.8.15 (Windows Store 1.8.49.0)
File Edit Sketch Tools Help

Hello_world $

#include <LiquidCrystal_I2C.h>
// Including the LCD Display Library

LiquidCrystal_I2C lcd = LiquidCrystal_I2C(0x27,16,2);
//Assigning the LCD address, rows and columns
void setup() {
    // put your setup code here, to run once:
    lcd.begin();
    lcd.backlight();
}

void loop() {
    // put your main code here, to run repeatedly:
    lcd.setCursor(0,0);
    int Variable_count,i;
    for(i=1;i<= 100; i++)
    {
        Variable_count = i;
        lcd.print(Variable_count);
        delay(1000);
        lcd.clear();
    }
}
```

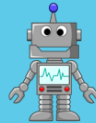
Figure 1.12

Workshop 3

- Adding Servo Motor



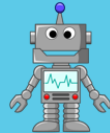
Hi Aebo, what are we doing today?



Andy, we are learning about how to use our servo motor



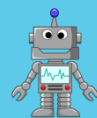
What is a servo motor?



A servomotor is a linear actuator that allows for precise control of angular or linear position which uses position feedback to control our robot's motion and final position



Are we adding the servo motor to our robot today?



Yes, let's start today's workshop



Step 1

First, take a servo motor and a servo horn. The servo has three cables:

- i) Brown → GND
- ii) Red → Vcc
- iii) Orange → Out

Step 2

Insert the servo horn with four sides on top of the motor and insert the motor into the board with wire going in first and connect it to the Servo (pin 8) port as shown in the figures. The side with the brown cable goes into the positive side of the servo motor pin located in the board

Refer to the following images for assembling the servo motor



Figure 1.15

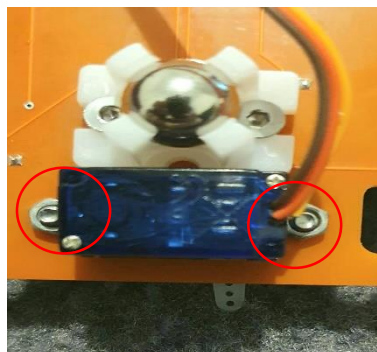


Figure 1.16

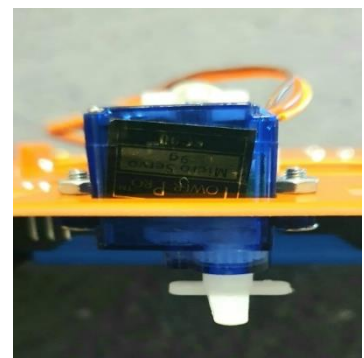


Figure 1.17

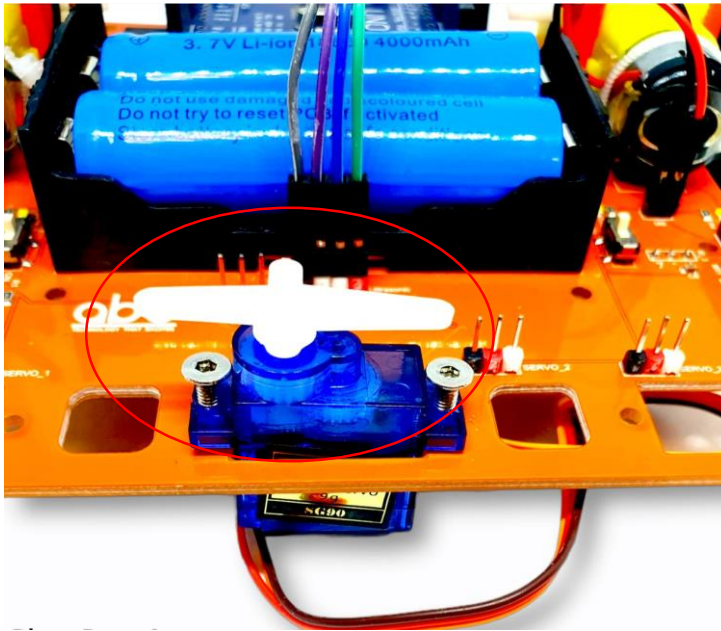


Figure 1.18

Step 3

To move the servo, first, upload the code in the following Figure into the Arduino. This code makes the servo move between angle 0 degrees and 180 degrees in the interval of 1 second. To move the servo, first, upload the code in the figure 1.19 to the Arduino. This code makes the servo move between angle 0 degrees and 180 degrees in the interval of 1 degree.

servo_1 | Arduino 1.8.15 (Windows Store 1.8.49.0)

File Edit Sketch Tools Help

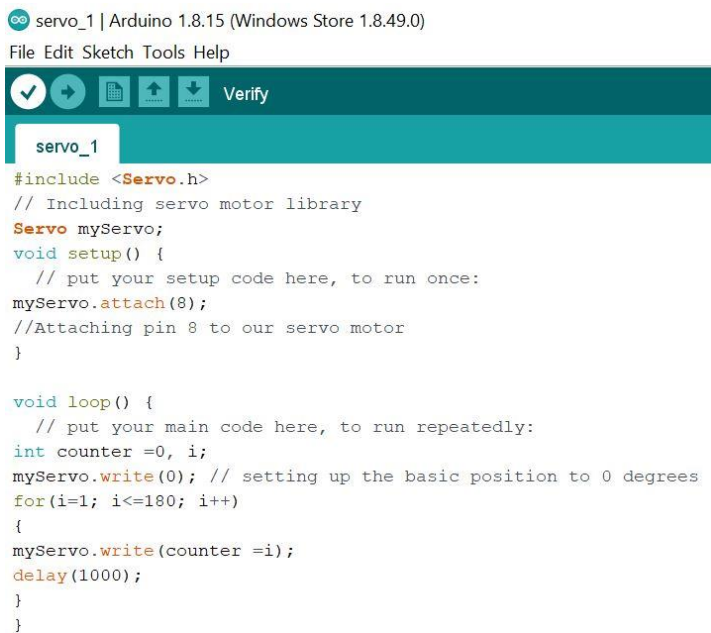
✓ → 📄 ⬆️ ⬆️ Verify

servo_1 \$

```
#include <Servo.h>
// Including servo motor library
Servo myServo;
void setup() {
    // put your setup code here, to run once:
    myServo.attach(8);
    //Attaching pin 8 to our servo motor
}

void loop() {
    // put your main code here, to run repeatedly:
    myServo.write(0);
    delay(1000);
    myServo.write(180);
    delay(1000);
}
```

Figure 1.19



```
servo_1 | Arduino 1.8.15 (Windows Store 1.8.49.0)
File Edit Sketch Tools Help

[Icons: Checkmark, Arrow, Upload, Download] Verify

servo_1

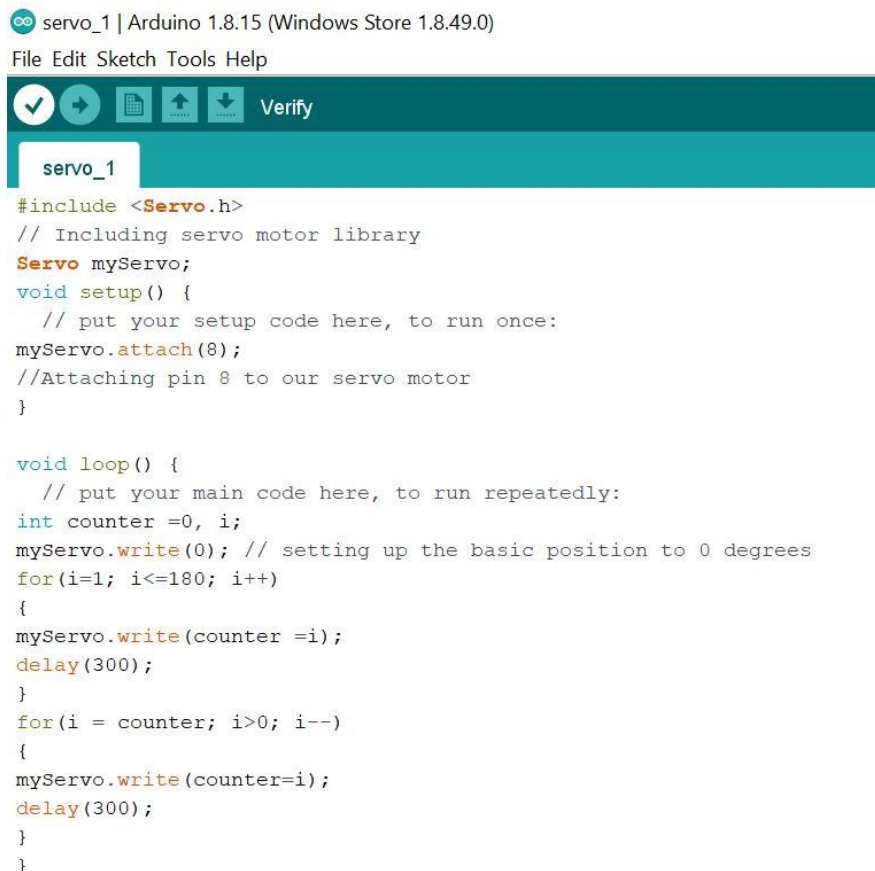
#include <Servo.h>
// Including servo motor library
Servo myServo;
void setup() {
  // put your setup code here, to run once:
  myServo.attach(8);
  //Attaching pin 8 to our servo motor
}

void loop() {
  // put your main code here, to run repeatedly:
  int counter =0, i;
  myServo.write(0); // setting up the basic position to 0 degrees
  for(i=1; i<=180; i++)
  {
    myServo.write(counter =i);
    delay(1000);
  }
}
```

Figure 1.20

Challenge 2

Make servo move in 10 degrees interval from 0 to 180 and 180 to 0



```
servo_1 | Arduino 1.8.15 (Windows Store 1.8.49.0)
File Edit Sketch Tools Help

[Icons: Checkmark, Arrow, Upload, Download] Verify

servo_1

#include <Servo.h>
// Including servo motor library
Servo myServo;
void setup() {
  // put your setup code here, to run once:
  myServo.attach(8);
  //Attaching pin 8 to our servo motor
}

void loop() {
  // put your main code here, to run repeatedly:
  int counter =0, i;
  myServo.write(0); // setting up the basic position to 0 degrees
  for(i=1; i<=180; i++)
  {
    myServo.write(counter =i);
    delay(300);
  }
  for(i = counter; i>0; i--)
  {
    myServo.write(counter=i);
    delay(300);
  }
}
```

Figure 1.21

Workshop 4

- Adding Ultrasonic sensor



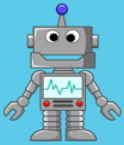
Hi Aebo, what are we doing today?



Andy, we are learning how to use our ultrasonic sensor



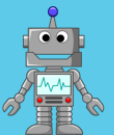
What is an ultrasonic sensor?



Ultrasonic sensors are devices that generate or sense ultrasound energy, it measures the distance of a target object by emitting ultrasonic sound waves, and converts the reflected sound into an electrical signal



Are we connecting the sensor to our Aebo?



Yes, let's start

Step 1

Insert the ultrasonic sensor into the casing

The next step is to connect the ultrasonic sensor with the mounting. Before doing this, upload the servo calibration program into your Arduino. This to make sure the servo motor is calibrated to face forward when the servo angle is 90

```
servo_1 | Arduino 1.8.15 (Windows Store 1.8.49.0)
File Edit Sketch Tools Help
[Checkmark] [Refresh] [Upload] Upload
servo_1 $
#include <Servo.h>
// Including servo motor library
Servo myServo;
void setup() {
  // put your setup code here, to run once:
  myServo.attach(8);
  //Attaching pin 8 to our servo motor
}

void loop() {
  // put your main code here, to run repeatedly:

  myServo.write(90); // setting up the basic position to 90 degrees
}
```

Figure 1.22



Figure 1.23



Figure 1.24

As shown in the figure, connect the ultrasonic sensor to the sensor mount and secure it using the screws given in the servo motor packet and connect the pins of ultrasonic sensor to the pins on the board using the jumper cables



Figure 1.26



Figure 1.25



Figure 1.27

To get the value of the distance in front of the robot use the code below. The distance is displayed in cms



```
distance_sensor | Arduino 1.8.15 (Windows Store 1.8.49.0)
File Edit Sketch Tools Help

distance_sensor $

#include <LiquidCrystal_I2C.h>
// Including the LCD Display Library
LiquidCrystal_I2C lcd = LiquidCrystal_I2C(0x27,16,2);
float calc_distance()
{
    digitalWrite(A1,LOW);
    delayMicroseconds(2);
    digitalWrite(A1, HIGH);
    delayMicroseconds(10);
    digitalWrite(A1,LOW);
    long int duration = pulseIn(A0, HIGH);
    float distance = duration * (0.034/2);
    return distance;
}

void setup() {
    // put your setup code here, to run once:
    lcd.begin();
    lcd.backlight();
    pinMode (A1, OUTPUT);
    pinMode (A2, INPUT);
}

void loop() {
    // put your main code here, to run repeatedly:
    lcd.setCursor(0,0);
    float distance = calc_distance();
    lcd.print(distance);
    delay(100);
    lcd.clear();
}
```

Figure 1.29

To complete this program, we have used basic mathematics principles like
 $\text{Speed} = \text{Distance} / \text{Time}$ and speed of light is 343m/s or 0.034 cm/s

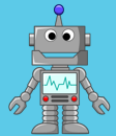
Workshop 5 & 6

- Social Distancing Robot



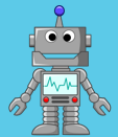
Hi Aebo, what are we doing today?

Andy, we are making our Aebo a social distancing Robot



What is a social distancing robot?

A social distancing robot will maintain safe distance from any obstacle, if an object appears in its path, it will stop and move backwards and after moving away from the obstacle, it goes forward and if the object maintains safe distance then the robot stays still



Sounds good, let's start coding then

We can use earlier defined blocks of forward, backward and stop Robot in our social distancing Robot

Final code for social distancing Robot

Social_Distancing | Arduino 1.8.15 (Windows Store 1.8.49.0)

File Edit Sketch Tools Help

```
✓ ↻ 📄 ⬆️ ⬇️

Social_Distancing $

#include<Servo.h>
#include<LiquidCrystal_I2C.h>
Servo myservo;
LiquidCrystal_I2C lcd(0x27,16,2);
int distance;
int duration;

void setup() {
  lcd.begin();
  pinMode(A0,INPUT);
  pinMode(A1,OUTPUT);
  pinMode(12,OUTPUT);
  pinMode(13,OUTPUT);
  pinMode(11,OUTPUT);
  pinMode(3,OUTPUT);
  myservo.attach(5);
  myservo.write(90);
}

void loop() {
  myservo.write(90);
  delay(100);
  lcd.setCursor(10,0);
  lcd.print(" ");
  calc_distance();
  lcd.setCursor(0,0);
  lcd.print("Distance");
  lcd.setCursor(10,0);
  lcd.print(distance);
  if (distance < 20)
  {
    backward();
  }
  if (distance > 20 and distance <35)
  {
    stopRobot();
  }
  if (distance > 35){
    forward();
  }
}
```

```
void calc_distance()
{
  digitalWrite(A1,LOW);
  delayMicroseconds(2);
  digitalWrite(A1,HIGH);
  delayMicroseconds(10);
  digitalWrite(A1,LOW);
  duration = pulseIn(A0,HIGH);
  distance = 0.034*duration/2;
}

void forward()
{
  digitalWrite(12,HIGH);
  digitalWrite(13,HIGH);
  analogWrite(3,100);
  analogWrite(11,100);
}

void backward()
{
  digitalWrite(12,LOW);
  digitalWrite(13,LOW);
  analogWrite(3,100);
  analogWrite(11,100);
}

void stopRobot()
{
  analogWrite(3,0);
  analogWrite(11,0);
}
```

Figure 1.30

Workshop 7 & 8

- Path finding Robot



Hi Aebo, what are we doing today?



Andy, we are making a path finding robot



What is a path finding robot?



A path finding robot travels through an obstacle course where humans cannot enter, for example robots used in mining, rescue robots used in underground caves



What are the steps to be followed?



We will learn the steps in detail along the way let's start

Follow the below steps to complete the path finding robot

- We need to program our robot so it moves forward when the distance in front of the robot is more than 20cms.
- If distance is less than 20cms, robot looks left and right.
- Gets the distance of each side.

Compares the distance and travel towards the direction which has greater distance

Final code for Path Finding Robot

```
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
float distance = 0;
float leftdistance = 0;
float rightdistance = 0;

Servo servo_8;

void seeleft () {
    servo_8.write(135);
}

void seeright () {
    servo_8.write(45);
}

void seestraight () {
    servo_8.write(90);
}

LiquidCrystal_I2C lcd = LiquidCrystal_I2C (0x27, 16, 2);

int calc_distance(int trig, int echo) {
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);
    float duration = pulseIn(echo, HIGH);
    int distanceValue = duration*0.034/2;
    return distanceValue;
}

void Forward () {
    digitalWrite(12, 1);
    digitalWrite(13, 1);
    analogWrite(11, 100);
    analogWrite(3, 100);
}
```

Figure 1.31

```

void left () {
    digitalWrite(12,1);
    digitalWrite(13,0);
    analogWrite(11,100);
    analogWrite(3,100);
}

void Backward () {
    digitalWrite(12,0);
    digitalWrite(13,0);
    analogWrite(11,100);
    analogWrite(3,100);
}

void right () {
    digitalWrite(12,0);
    digitalWrite(13,1);
    analogWrite(11,100);
    analogWrite(3,100);
}

void StopRobot () {
    digitalWrite(12,0);
    digitalWrite(13,0);
    analogWrite(11,0);
    analogWrite(3,0);
}

void setup() {
    servo_8.attach(8);
    lcd.begin();
    pinMode(A0, INPUT);
    pinMode(A1, OUTPUT);
    pinMode(12, OUTPUT);
    pinMode(13, OUTPUT);
    pinMode(11, OUTPUT);
    pinMode(3, OUTPUT);
}

void loop() {
    lcd.setCursor(0,0);
    lcd.print("Distance:");
    lcd.setCursor(10,0);
    distance = calc_distance(A1,A0);
    lcd.print(distance);

    if(distance < 20) {
        lcd.setCursor(0,1);
        lcd.print("Obstacle");
        seeleft();
        delay(500);
        leftdistance = calc_distance(A1,A0);
        seeright();
        delay(500);
        rightdistance = calc_distance(A1,A0);
        seestraight();
        if(leftdistance > rightdistance) {
            left();
            delay(500);
        }
        else {
            right();
            delay(500);
        }
    }
    else {
        lcd.setCursor(0,1);
        lcd.print("Go Forward");
        Forward();
    }
}

```

Figure 1.32